Exp No: 3	**Training with Stochastic Gradient Descent (SGD) with Momentum and Comparison with the Adam Optimizer**

AIM:

To implement a training algorithm using stochastic gradient descent (SGD) with momentum, train a model with it and with the Adam optimizer on the same dataset, and compare their convergence rates and performance.

ALGORITHM / PROCEDURE:

1. Load the handwritten digits dataset (1797 images of 8x8 pixels), scale it to 0-1 and split it 80% / 20% into training and test sets.
2. Build a neural network (64 inputs, 64 hidden neurons with *ReLU*, 10 softmax outputs) using NumPy, with cross-entropy loss. Both models start from the **same initial weights**.
3. **SGD with momentum:** keep a velocity `v = 0.9*v - lr*grad` and update `w = w + v` (lr = 0.05).
4. **Adam:** keep running averages of the gradient and squared gradient, correct their bias, and update `w = w - lr*m_hat/(sqrt(v_hat)+eps)` (lr = 0.005).
5. Train both models for 60 epochs with mini-batches of 32 and record the training loss and accuracy after every epoch.
6. Compare the number of epochs needed to reach a training loss below 0.1, and the final test loss and accuracy.
7. Plot the loss and accuracy curves of both optimizers.

Fig 1: Neural network trained with both optimizers

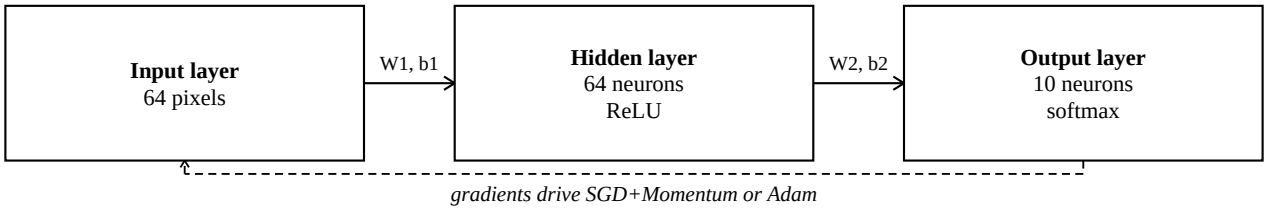
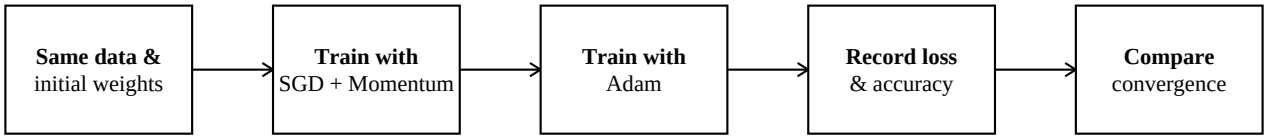


Fig 2: Comparison procedure



Commands / Settings used:

Setting	SGD with Momentum	Adam
Learning rate	0.05	0.005
Momentum / betas	momentum = 0.9	beta1 = 0.9, beta2 = 0.999
Batch size / epochs	32 / 60	32 / 60
Dataset	scikit-learn digits (8x8)	scikit-learn digits (8x8)
Loss function	Cross-entropy	Cross-entropy

PROGRAM:

```
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

# ---------- Data: handwritten digits (8x8 images) ----------
d = load_digits()
X = d.data / 16.0; y = d.target
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
Ytr = np.eye(10)[ytr]; Yte = np.eye(10)[yte]

def init(seed=0):
    r = np.random.RandomState(seed)
    return [r.randn(64, 64) * np.sqrt(2 / 64), np.zeros(64),
            r.randn(64, 10) * np.sqrt(2 / 64), np.zeros(10)]

def forward(p, X):
    h = np.maximum(0, X @ p[0] + p[1])
    z = h @ p[2] + p[3]
    e = np.exp(z - z.max(1, keepdims=True))
    return h, e / e.sum(1, keepdims=True)

def loss_fn(P, Y):
    return -np.mean(np.sum(Y * np.log(P + 1e-12), 1))

def grads(p, xb, yb):
    h, P = forward(p, xb)
    d2 = (P - yb) / len(xb)
    d1 = (d2 @ p[2].T) * (h > 0)
    return [xb.T @ d1, d1.sum(0), h.T @ d2, d2.sum(0)]

def train(optim, epochs=60, batch=32, seed=0):
    p = init(); r = np.random.RandomState(seed)
    v = [np.zeros_like(a) for a in p]; m = [np.zeros_like(a) for a in p]; t = 0
    hist = {"loss": [], "tacc": [], "teacc": []}
    for ep in range(epochs):
        idx = r.permutation(len(Xtr))
        for s in range(0, len(idx), batch):
            b = idx[s:s + batch]
            g = grads(p, Xtr[b], Ytr[b]); t += 1
            for i in range(4):
                if optim == "sgd_momentum":            # SGD with momentum
                    v[i] = 0.9 * v[i] - 0.05 * g[i]
                    p[i] += v[i]
                else:                                  # Adam
                    m[i] = 0.9 * m[i] + 0.1 * g[i]
                    v[i] = 0.999 * v[i] + 0.001 * g[i] ** 2
                    mh = m[i] / (1 - 0.9 ** t); vh = v[i] / (1 - 0.999 ** t)
                    p[i] -= 0.005 * mh / (np.sqrt(vh) + 1e-8)
        _, P = forward(p, Xtr); _, Pt = forward(p, Xte)
        hist["loss"].append(loss_fn(P, Ytr))
        hist["tacc"].append(np.mean(P.argmax(1) == ytr) * 100)
        hist["teacc"].append(np.mean(Pt.argmax(1) == yte) * 100)
    return hist, loss_fn(Pt, Yte)
```

```
H1, tl1 = train("sgd_momentum"); H2, tl2 = train("adam")

print(f"{'Epoch':<7}{'SGD+Mom loss':<14}{'SGD+M train acc':<16}"
      f"{'Adam loss':<11}{'Adam train acc'}")
print("-" * 56)
for e in [1, 2, 5, 10, 20, 40, 60]:
    print(f"{e:<7}{H1['loss'][e-1]:<14.4f}{H1['tacc'][e-1]:<16.2f}"
          f"{H2['loss'][e-1]:<11.4f}{H2['tacc'][e-1]:.2f}")

def reach(h, thr=0.1):
    return next((i + 1 for i, l in enumerate(h["loss"]) if l < thr), None)
print("\nEpochs to reach training loss < 0.1 : "
      f"SGD+Momentum = {reach(H1)}, Adam = {reach(H2)}")
print("Final test accuracy : "
      f"SGD+Momentum = {H1['teacc'][-1]:.2f}%, Adam = {H2['teacc'][-1]:.2f}%")
print(f"Final test loss     : SGD+Momentum = {tl1:.4f}, Adam = {tl2:.4f}")

# ---------- Convergence plot ----------
fig, ax = plt.subplots(1, 2, figsize=(9, 3.4))
ep = np.arange(1, 61)
ax[0].plot(ep, H1["loss"], label="SGD + Momentum"); ax[0].plot(ep, H2["loss"], label="Adam")
ax[0].set_yscale("log")
ax[0].set_xlabel("Epoch")
ax[0].set_ylabel("Training loss")
ax[0].set_title("Loss vs epoch")
ax[0].legend()
ax[1].plot(ep, H1["teacc"], label="SGD + Momentum"); ax[1].plot(ep, H2["teacc"], label="Adam")
ax[1].set_xlabel("Epoch")
ax[1].set_ylabel("Test accuracy (%)")
ax[1].set_title("Test accuracy vs epoch")
ax[1].legend()
for a in ax: a.grid(alpha=.3)
plt.tight_layout(); plt.savefig("img/exp3_curves.png", dpi=200)
```

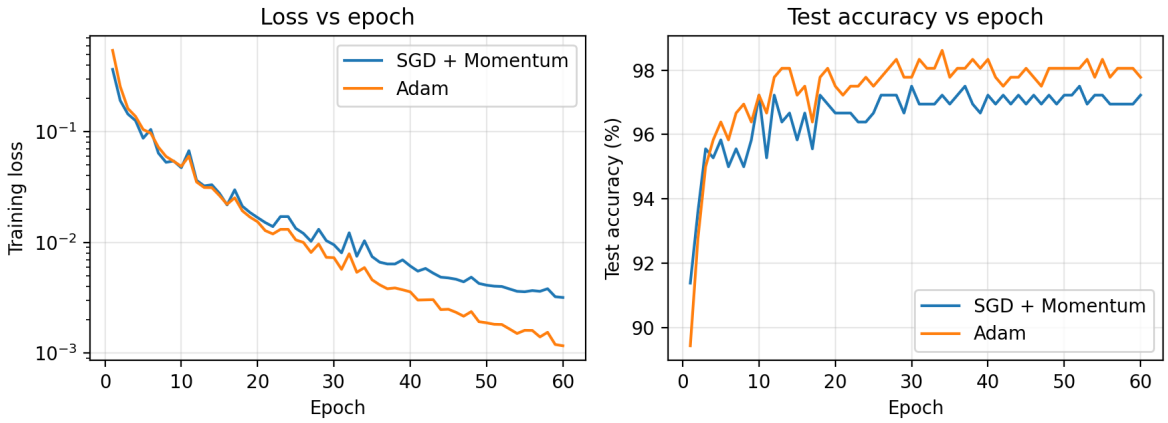
OUTPUT:

Terminal - SGD with momentum vs Adam

```
root@ip-10-10-5-10:~# python3 exp3.py
Epoch  SGD+Mom loss  SGD+M train acc Adam loss  Adam train acc
-----------------------------------------------------------
1      0.3654        90.40           0.5423     90.33
2      0.1910        94.71           0.2556     94.08
5      0.0873        97.36           0.1047     97.84
10     0.0473        99.23           0.0487     98.96
20     0.0168        99.86           0.0153     99.93
40     0.0061        100.00          0.0036     100.00
60     0.0032        100.00          0.0012     100.00

Epochs to reach training loss < 0.1 : SGD+Momentum = 5, Adam = 6
Final test accuracy : SGD+Momentum = 97.22%, Adam = 97.78%
Final test loss     : SGD+Momentum = 0.0757, Adam = 0.0646
```

Fig 3: Convergence of SGD with Momentum and Adam



RESULT:

Thus SGD with momentum was implemented and compared with the Adam optimizer on the same network and data. SGD with momentum reduced the loss slightly faster in the first epochs (training loss below 0.1 after 5 epochs against 6 for Adam), while Adam reached a lower final training loss (0.0012 against 0.0032) and a slightly better test result (97.78% accuracy, test loss 0.0646, against 97.22% and 0.0757). Both optimizers converged well on this small dataset.